

Part 1: From 1,540 to 19,511 Tokens per Second on a Single TPU Chip

A Measurement Study of Large Language Model Inference Optimization on
TPU v5e

*Throughput, latency, energy, and cost results for Gemma 2B served with vLLM, with
observations on the JAX and Pallas compilation stack*

Technical and Economic Whitepaper
July 2026

Rubens de Almeida Zimbres
Google Developer Expert in AI and Cloud
Email: rubenszmm@gmail.com

*Google ML Developer Programs and Google Developers Program supported this work by providing Google
Cloud Credits*

Executive Summary

This paper reports a measurement study of large language model inference on a single Google TPU v5e chip. The subject model was Gemma 2B, served through the vLLM inference engine on the JAX based TPU software stack. The study asked one question: *how much of the chip's potential does a default serving configuration leave unused, and how far can disciplined measurement recover it without changing the model or hardware.*

The answer was a factor of 12.7. The starting configuration produced 1,540 tokens per second; the final configurations produced 16,700 at the recommended operating point and 19,511 at the throughput maximum. The gain came from three interventions, each identified by profiling the chip rather than by guesswork: a change to how output tokens are selected, worth 2.3 times on its own; a systematic increase in the number of requests processed together, worth a further 4.3 times; and a retuning of the attention kernel's block size to match the workload's sequence lengths, delivered through a custom Pallas kernel harness, worth a further 24 percent. No model weights were changed, no accuracy was sacrificed, and no additional hardware was used. Because the chip and its power envelope stayed constant, every efficiency metric improved by the same factor: the energy bound per token fell from roughly 128 millijoules to 10, and at published on demand pricing the cost of generation fell from roughly 4.6 million tokens per dollar to 58.5 million.

The study is equally defined by what did not work, and those results are reported with the same care. Speculative decoding, a widely promoted technique, reduced throughput by a factor of six in this workload's operating regime. Weight and cache quantization turned out to be unavailable on this software stack for this model, a finding documented at the source level. A precision optimization that looked certain on paper reduced throughput by 14 percent when applied, and was reverted. Each negative result was verified before being accepted, and each carries a practical lesson.

This document describes what was done, why, and with what results. The specific procedures, code, and integration techniques are not disclosed.

1. Background and Motivation

Serving a language model is a different engineering problem from training one. Training is dominated by arithmetic. Serving, and in particular the token by token generation phase called decode, is dominated by data movement: for every token generated, the chip must read essentially the entire model from its high bandwidth memory. On the chip studied here, that memory delivers 819 gigabytes per second and the model occupies roughly five gigabytes in its standard 16 bit form, two numbers that set a hard ceiling on generation speed no software can exceed. The question for any deployment is how close to that ceiling it operates.

Most deployments do not know the answer, because default framework configurations are tuned for generality rather than for a specific model on a specific chip. This study set out to find the answer for one concrete case: Gemma 2B, a two and a half billion parameter open model from Google, served with vLLM, the most widely used open source inference engine, on a single TPU v5e chip, the efficiency oriented member of Google's TPU family, in a standard hosted notebook environment reproducible by any practitioner with the same access.

The choice of a small model on a single chip was deliberate. Small models are increasingly the economic workhorses of production systems, handling classification, extraction, routing, and drafting tasks where a frontier model would be wasteful. For these workloads, tokens per second per dollar decides profitability, and a single chip is the unit at which that metric is set; whatever efficiency is achieved on one chip multiplies directly across a fleet. The study also had a second purpose. The TPU serving stack for vLLM is new, built on a hardware plugin that unifies JAX and PyTorch execution paths and compiles model code through XLA, with performance critical kernels written in Pallas, a JAX embedded language for custom TPU and GPU kernels. New stacks have gaps, and part of the value of a measurement campaign is mapping where the advertised surface and the implemented surface differ. Several such gaps were found and are reported here.

2. What Was Done

The method of the study can be stated in one sentence: measure first, predict second, change third, and verify fourth. Every intervention followed the same loop: a hardware profile identified where device time was actually spent, at the level of individual compiled operations; a specific change was predicted to produce a specific gain; the change was applied, one variable at a time; and the result was compared against the prediction. Changes that failed their prediction were reverted and recorded, not rationalized. Three categories of change were explored: the token selection path, the batching configuration, and the byte reduction category covering weight quantization, cache quantization, and numeric precision in the output pipeline. In parallel, a custom attention kernel written in Pallas was integrated into the production serving path and validated against the stock implementation, establishing a harness for future kernel work.

Two disciplines were applied throughout. First, every measured number required a paired verification: a second counter, a repeated run, or a trace level confirmation that the intended change had actually taken effect. On three occasions this caught a configuration that had been requested but silently not applied, which would otherwise have produced false conclusions. Second, headline numbers were always taken with profiling disabled, since tracing adds overhead; profiled runs were used only to rank operations. The workload for all measurements was a fixed set of short prompts generating 128 tokens each under deterministic selection, executed as an offline batch. This is a decode heavy, short context workload; its limitations are discussed in Section 9.

3. Results: The Throughput Campaign

3.1 The two interventions that worked

Profiling the starting configuration produced a surprise. The attention computation, the usual suspect in transformer performance work, accounted for only a small share of device time; the largest consumer, at roughly 55 percent, was the token selection machinery, executing an expensive search procedure on every step to support a sampling feature the workload was not meaningfully using. Reconfiguring the request so that selection took the direct path removed that cost entirely: throughput rose from roughly 1,540 to 3,577 tokens per second, a factor of 2.3, with no change to the model.

With the selection cost gone, the remaining time was dominated by operations whose duration is set by memory traffic: reading the feed forward weights and the large output projection over the model's 256,000 entry vocabulary. Such operations have a decisive property: their cost per step is fixed regardless of how many requests share the step, so doubling the concurrent requests nearly doubles throughput. The study followed this logic up a doubling ladder, verifying at each rung that the predicted operations stayed fixed in cost while others grew.

Concurrent requests	Tokens per second	Latency per batch (ms)	Batch time to first token (ms)	Tokens per second per rated watt
32	3,577	8.4	84	18.2
64	6,449	8.9	141	32.7
128	9,790	11.9	157	49.7
256	13,470	17.7	187	68.4
512	15,338	31.2	307	77.9

Table 1. Batch scaling results, clean timing runs with profiling disabled. The watt figure uses the vendor reported 197 watt rating as an upper bound (Section 7).

```
(EngineCore pid=4426) INFO 07-04 15:08:44 [compilation_manager.py:133] Precompile structured_decode --> {'num_reqs': 256
(EngineCore pid=4426) INFO 07-04 15:08:45 [compilation_manager.py:167] Compilation of structured_decode --> {'num_reqs':
(EngineCore pid=4426) INFO 07-04 15:08:45 [compilation_manager.py:207] Warm-up call pass finished in 0.01 [secs] over 6
(EngineCore pid=4426) INFO 07-04 15:08:45 [core.py:306] init engine (profile, create kv cache, warmup model) took 184.21
(EngineCore pid=4426) WARNING 07-04 15:08:45 [vllm.py:1097] Inductor compilation was disabled by user settings, optimiza
(EngineCore pid=4426) INFO 07-04 15:08:45 [kernel.py:270] Final IR op priority after setting platform defaults: IrOpPrio
(EngineCore pid=4426) INFO 07-04 15:08:45 [tpu_platform.py:229] Initialized sharding configuration: ShardingConfigManage
(EngineCore pid=4426) INFO 07-04 15:08:45 [tpu_platform.py:316] Force using UniProcExecutor for JAX on single host witho
WARNING 07-04 15:08:45 [tpu_platform.py:395] Pin memory is not supported on TPU.
Rendering prompts: 100% 2/2 [00:00<00:00, 841.38it/s]
Processed prompts: 100% 2/2 [00:00<00:00, 5.37it/s, est. speed input: 32.23 toks/s, output: 42.97 toks/s]
Rendering prompts: 100% 256/256 [00:00<00:00, 3472.16it/s]
Processed prompts: 100% 256/256 [00:00<00:00, 1839.67it/s, est. speed input: 11047.79 toks/s, output: 1841.24 toks/s]
Rendering prompts: 100% 256/256 [00:00<00:00, 4058.78it/s]
Processed prompts: 100% 256/256 [00:02<00:00, 108.45it/s, est. speed input: 650.72 toks/s, output: 13882.04 toks/s]

===== METRICS =====
Spec decode: OFF
Profiler: OFF
Engine init (weights+compile): 276.4 s
Warmup generate: 0.38 s
Batch TTFT (prefill+1 tok): 213 ms [offline, batch=256]
Generation: 32768 tokens in 2.42 s = 13,517.5 tok/s
Inter-token latency (approx): 17.4 ms/token/seq
Time to first result (cold): 280.3 s [script start -> now]
Spec-based efficiency: 68.6 tok/s per rated W (~15 mJ/token upper bound)
=====
```

```

(EngineCore pid=43785) INFO 07-05 16:55:09 [compilation_manager.py:167] Compilation of structured_decode --> {'num_reqs': 16} finis
(EngineCore pid=43785) INFO 07-05 16:55:10 [compilation_manager.py:133] Precompile structured_decode --> {'num_reqs': 32}
(EngineCore pid=43785) INFO 07-05 16:55:10 [compilation_manager.py:167] Compilation of structured_decode --> {'num_reqs': 32} finis
(EngineCore pid=43785) INFO 07-05 16:55:10 [compilation_manager.py:133] Precompile structured_decode --> {'num_reqs': 64}
(EngineCore pid=43785) INFO 07-05 16:55:10 [compilation_manager.py:167] Compilation of structured_decode --> {'num_reqs': 64} finis
(EngineCore pid=43785) INFO 07-05 16:55:10 [compilation_manager.py:133] Precompile structured_decode --> {'num_reqs': 128}
(EngineCore pid=43785) INFO 07-05 16:55:11 [compilation_manager.py:167] Compilation of structured_decode --> {'num_reqs': 128} finis
(EngineCore pid=43785) INFO 07-05 16:55:11 [compilation_manager.py:133] Precompile structured_decode --> {'num_reqs': 256}
(EngineCore pid=43785) INFO 07-05 16:55:11 [compilation_manager.py:167] Compilation of structured_decode --> {'num_reqs': 256} finis
(EngineCore pid=43785) INFO 07-05 16:55:11 [compilation_manager.py:133] Precompile structured_decode --> {'num_reqs': 512}
(EngineCore pid=43785) INFO 07-05 16:55:11 [compilation_manager.py:167] Compilation of structured_decode --> {'num_reqs': 512} finis
(EngineCore pid=43785) INFO 07-05 16:55:11 [compilation_manager.py:207] Warm-up call pass finished in 0.01 [secs] over 7 tasks.
(EngineCore pid=43785) INFO 07-05 16:55:11 [core.py:306] init engine (profile, create kv cache, warmup model) took 144.32 s (compil
(EngineCore pid=43785) WARNING 07-05 16:55:12 [vllm.py:1097] Inductor compilation was disabled by user settings, optimizations sett
(EngineCore pid=43785) INFO 07-05 16:55:12 [kernel.py:270] Final IR op priority after setting platform defaults: IrOpPriorityConfig
(EngineCore pid=43785) INFO 07-05 16:55:12 [tpu_platform.py:229] Initialized sharding configuration: ShardingConfigManager(total_de
(EngineCore pid=43785) INFO 07-05 16:55:12 [tpu_platform.py:316] Force using UniProcExecutor for JAX on single host without pipelin
WARNING 07-05 16:55:12 [tpu_platform.py:395] Pin memory is not supported on TPU.
Rendering prompts: 100% ██████████ 2/2 [00:00<00:00, 877.65it/s]
Processed prompts: 100% ██████████ 2/2 [00:00<00:00, 7.14it/s, est. speed input: 42.86 toks/s, output: 57.14 toks/s]
Rendering prompts: 100% ██████████ 512/512 [00:00<00:00, 4847.79it/s]
Processed prompts: 100% ██████████ 512/512 [00:00<00:00, 2648.71it/s, est. speed input: 15903.03 toks/s, output: 2650.43 toks/s]
Rendering prompts: 100% ██████████ 512/512 [00:00<00:00, 4926.36it/s]
Processed prompts: 100% ██████████ 512/512 [00:03<00:00, 157.32it/s, est. speed input: 943.98 toks/s, output: 20138.20 toks/s]

===== METRICS =====
Spec decode: OFF
Profiler: OFF
Engine init (weights+compile): 173.9 s
Warmup generate: 0.28 s
Batch TTFT (prefill+1 tok): 300 ms [offline, batch=512]
Generation: 65536 tokens in 3.36 s = 19,511.2 tok/s
Inter-token latency (approx): 24.1 ms/token/seq
Time to first result (cold): 178.0 s [script start -> now]
Spec-based efficiency: 99.0 tok/s per rated W (~10 mJ/token upper bound)
=====

```

The scaling factors tell the story precisely. Doubling from 32 to 64 requests returned 1.81 times the throughput, nearly the ideal 2.0, confirming the chip had been waiting on memory rather than computing. Each further doubling returned less: 1.50, then 1.38, then 1.13. The diminishing curve was predicted in advance, because two categories of work do grow with the number of requests: the attention computation, whose cache reads are proportional to requests served, and the output post processing. At 512 concurrent requests the crossover was measured directly: the attention family had become the single largest consumer of the step, and further batching bought almost nothing. The recommended operating point is 256 concurrent requests, which captures 88 percent of maximum throughput at 17.7 milliseconds per token, against 31.2 at the maximum. Throughput oriented deployments can run at 512; interactive services will prefer 256 or below. The value of the table is that the trade is quantified rather than assumed.

3.2 The overall arc

Stage	Configuration	Tokens per second	Versus start
Start	Default selection path, 32 requests	~1,540	1.0x
Selection fix	Direct token selection, 32 requests	3,577	2.3x
Batch scaling	256 requests (recommended point)	13,470	8.7x
Batch scaling	512 requests (throughput maximum)	15,338	10.0x
Kernel tuning	Attention block size 256, 256 requests	16,700	10.8x
Kernel tuning	Attention block size 256, 512 requests	19,511	12.7x

Table 2. The campaign in four rows. Two interventions account for the entire gain; the chip, the model, and its outputs are unchanged throughout.

A separate verification anchors the credibility of the numbers above. The 256 request configuration was measured in three independent sessions, on rebuilt virtual machines, returning 13,517, 13,470, and 13,484 tokens per second, a spread of one third of one percent; one outlier session returned 12,991, a 3.6 percent dip attributed to environment variance, reported here rather than discarded.

The headline is therefore quoted as approximately 13.5 thousand tokens per second with a known spread, not as a single lucky run.

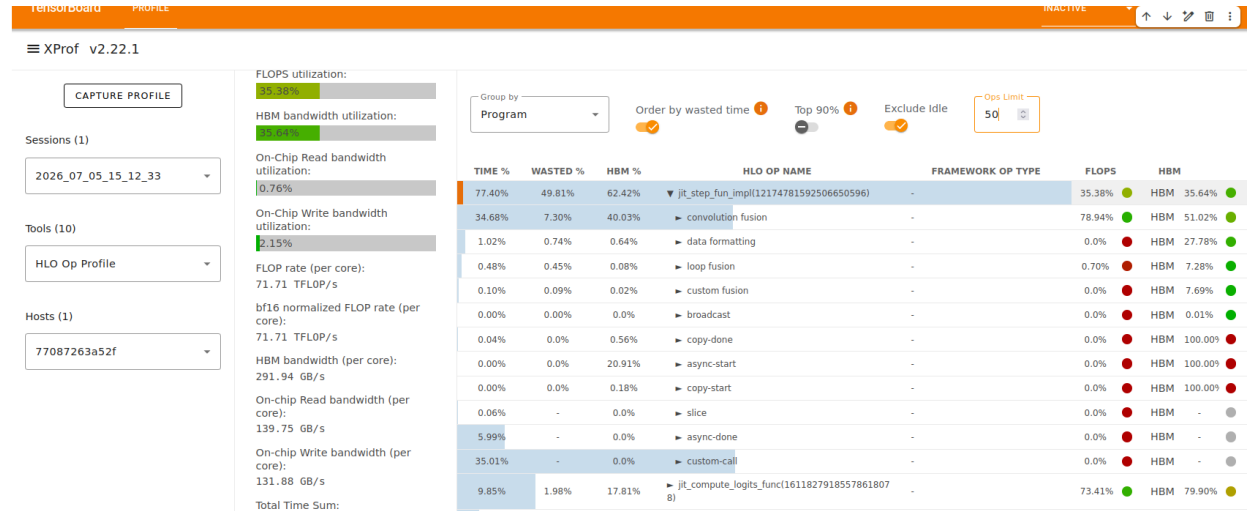
4. What the Measurements Revealed About the Chip

Three utilization questions matter for a serving deployment, and the study answered each with a direct measurement rather than an estimate.

First, is the chip idle. Trace analysis found roughly 85 microseconds of device silence per 11.9 millisecond generation step at 128 concurrent requests, which is 0.7 percent. The scheduler and host side machinery keep the accelerator more than 99 percent occupied even at high concurrency; host overhead is not a bottleneck in this stack, and the study can say so with a number rather than an impression.

Second, how hard is the memory system working. The two dominant weight reading operations were measured at 91 and 84 percent of the chip's theoretical memory bandwidth while executing. These operations are finished as optimization targets in their current numeric format; only reducing the bytes themselves could make them faster. Averaged across a whole step, memory utilization is roughly 40 percent, because part of each step goes to operations limited by other factors.

Third, how hard is the arithmetic hardware working. Arithmetic utilization during generation is roughly a third of the chip's 197 teraFLOP per second bfloat16 peak. Quoted alone, that would suggest waste. It does not. Generation is memory bound by nature; the arithmetic units spend most of their time waiting for weights to arrive, and modest arithmetic utilization alongside high memory utilization is the expected signature of a well tuned decode workload. During the prompt processing phase, which is arithmetic bound, the same chip measured 180 to 190 teraFLOPs per second, above 90 percent of peak, confirming the hardware performs when the workload demands it.



The trace work also produced a finding of general value: which operations are free to batch and which are not. Across an eightfold increase in concurrent requests, the feed forward weight read measured 183, 183, and then 375 microseconds, flat until the arithmetic itself began to matter at the largest batch. The attention operation over the same range measured 89.6, 179.1, and 714.4 microseconds, growth of 7.97 times against an 8 times batch increase, linear to within measurement error, while the output projection stayed flat. This comparison, predicted in advance and confirmed at three operating points, explains the entire shape of the scaling curve in Table 1 and generalizes to any transformer on any memory bound accelerator: batching is nearly free until attention and per request bookkeeping catch up, and the crossover is discoverable by measurement.

4.1 Arithmetic intensity

The single quantity behind all of these regimes is arithmetic intensity, defined for any operation as

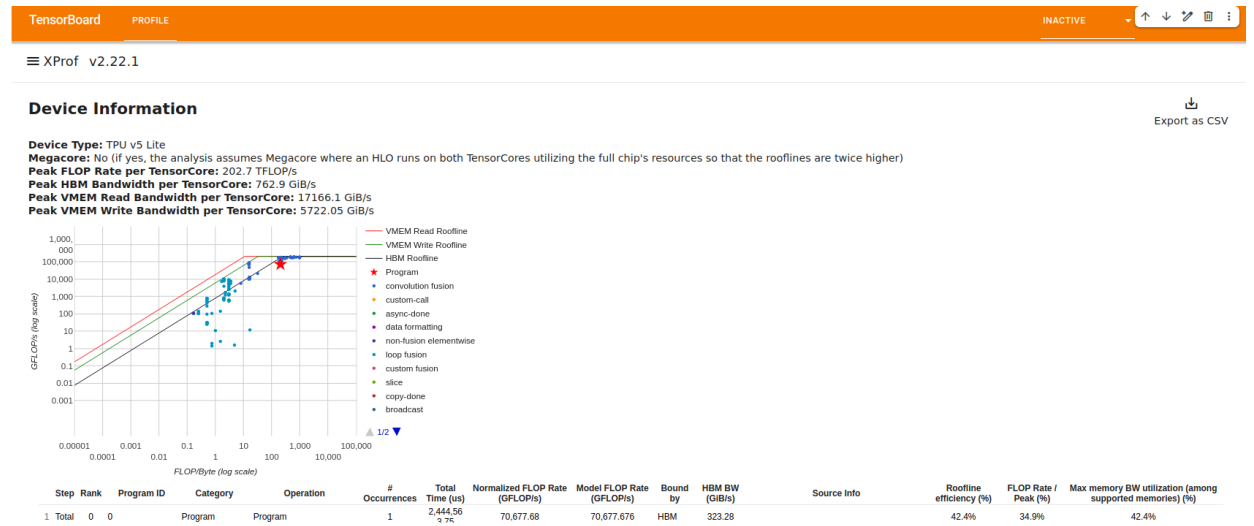
$$\text{Arithmetic Intensity} = \text{Computation FLOPs} \div \text{Communication Bytes}.$$

The accelerator has an intensity of its own: peak arithmetic rate over memory bandwidth, here 1.97e14 FLOPs per second over 8.19e11 bytes per second, about 240 FLOPs per byte. An operation below 240 cannot keep the arithmetic units fed and is memory bound; above 240 it is compute bound. Because a decode matmul multiplies a batch of B token vectors against a fixed weight matrix, its intensity is approximately B, which turns the batching campaign into one prediction: the weight streaming operations should stay memory bound, and flat in cost, until the batch approaches 240 tokens per step, and become compute bound beyond it. Table 3 computes the intensities from the model's dimensions and checks them against the measured times.

Operation	Intensity at batch 32	128	256	512	Measured behavior
Feed forward projection	31	120	226	404	flat at 183 us through 128; 375 us at 512 (roofline predicts 349)
Output projection (256k vocab)	31	120	227	409	near flat to 1.66 ms; 2.80 ms at 512 (roofline predicts 2.72)
Attention (decode)	8	8	8	8	always memory bound; cost linear in batch, 7.97x over an 8x increase

Table 3. Arithmetic intensity in FLOPs per byte, derived from the model's published dimensions in bfloat16, against the chip's critical intensity of about 240. The measured times of Section 4 sit within ten percent of the corresponding roofline on both sides of the threshold.

The table explains three campaign results at once. The weight streaming operations cross the 240 threshold between batch 256 and 512, exactly where their measured cost stopped being flat, which is why batch 256 is the recommended operating point: the last rung of the ladder on the memory bound side of the roofline. Attention's intensity is about eight regardless of batch, because eight query heads share one key value head and each request reads its own cache; an operation that can never amortize its bytes across the batch is precisely one whose cost grows linearly with it. And the same threshold predicts where quantization would pay: halving the weight bytes doubles an operation's intensity, moving the crossover to roughly batch 120 and halving the memory bound cost below it.



5. What Did Not Work, and Why That Matters

5.1 Speculative decoding

Speculative decoding, in which a cheap mechanism drafts several tokens ahead and the model verifies them in one pass, is frequently promoted with claims of three to four times speedups. Applied to this workload at 32 concurrent requests, it reduced throughput from roughly 3,555 to 607 tokens per second, a loss of a factor of six. The explanation is regime dependence: speculation spends spare compute to reduce the latency of individual requests, a good trade when the chip has idle capacity, as it does at very small batch sizes. At meaningful batch sizes the chip has no spare capacity; every verified draft displaces real work and rejected drafts are pure waste. It is a latency tool being sold as a throughput tool. The lesson: published speedups carry an implicit operating regime, and the only defense is measuring in one's own regime, which takes a single run.

5.2 Quantization

With the bandwidth ceiling measured and nearly reached, the remaining structural lever was reducing the bytes themselves: storing weights in 8 bits instead of 16, halving traffic on exactly the operations measured at 84 to 91 percent of bandwidth. The study attempted this through every

route the stack advertises, and the results form a finding of their own. The integer 8 bit route was rejected with an explicit error: accepted by the platform's configuration surface, not implemented in its model loader for this model family. The floating point 8 bit route for weights requires a checkpoint quantized in advance and fails unhelpfully when given a standard one; there is no on the fly path. The 8 bit route for the attention cache was the most instructive: the flag was accepted end to end, the engine ran, and the cache was allocated in 16 bits anyway, with no warning; only a direct check of the allocated cache size revealed that nothing had happened. Three attempts produced one loud failure, one unhelpful failure, one silent failure, and zero functioning quantization paths for this model on this stack version. This is reported as a map, not criticism; the stack is new and openly labels parts of this machinery as incomplete. The general lesson is sharp: on a fast moving inference stack, a configuration flag being accepted is not evidence that it did anything. Verify the allocation, the memory footprint, or the trace, not the flag.

5.3 The precision experiment

Trace analysis identified a genuine inefficiency in the output pipeline: on every generation step, the model's output scores are converted to 32 bit precision and copied, a cost that grows with batch size and reached roughly nine percent of the step at the largest batch. Under deterministic selection this precision is mathematically unnecessary, and the selection code demonstrably computes its answer before the conversion happens; removing it looked certain to return several percent. It did the opposite: applied as a minimal, correctness preserving change, it reduced throughput by 14 percent and multiplied time to first token by five, reproducibly, and was reverted with the baseline reconfirmed. The most probable explanation is that the conversion, wasteful in isolation, was load bearing for the surrounding compiled pipeline, which had been specialized around 32 bit outputs; changing the type at one point forced the runtime off its optimized path downstream. The lesson is among the most valuable in the study: on a just in time compiled stack, operation level arithmetic does not compose across compilation boundaries, and a change that removes a measured cost can add a larger unmeasured one. The inefficiency is real and has been documented for the stack's maintainers, where it can be fixed at the pipeline level.

6. Custom Kernels, Pallas, and the Compilation Stack

6.1 The Pallas kernel work

Part of the study integrated a custom attention kernel, written in Pallas, into the live serving path of the engine, replacing the stock implementation. The integration was verified twice over: by instrumentation proving the custom code executed inside the engine's core process, and by hardware traces attributing the attention operation's device time to the custom kernel's source file across multiple sessions and batch sizes. The measured result of the substitution was, by design, no change in performance; that equality was the success condition, because a substitution that changed results before any intentional modification would be measuring integration error, not kernel quality.

To be direct about what was and was not claimed at the validation stage: the unmodified substitution produced no throughput change, and no comparison in that phase should be read as Pallas outperforming the XLA compiled path. What the kernel work established was the platform for gains. The traces showed the attention kernel to be the one major operation with substantial headroom: it ran at roughly one quarter of memory bandwidth in this short context regime, dominated by per request overhead rather than data movement, and grew to the largest consumer of the step at scale, roughly 41 percent of device time at 512 concurrent requests. A validated, hot swappable custom kernel sitting on exactly that operation was the study's main asset, and it was then used.

The stock kernel walks each request's key value history in blocks whose size is chosen by a heuristic that targets enough bytes per block to saturate memory bandwidth, about 16 megabytes, which resolves to blocks of 2,048 tokens under this model's configuration. The workload's sequences average about 140 tokens, so roughly 93 percent of every block the kernel processed was masked padding: loaded, computed, and discarded. A small change to the vendored kernel made the block size configurable and set it to 256 tokens. Block size affects scheduling, not arithmetic, so the generated outputs are identical by construction, and the change carries its own verification, because the compiled kernel's name encodes its block configuration and must show the new size in any hardware trace.

The result was measured with the full protocol. In a same session comparison at 256 concurrent requests, throughput rose from 13,479 to 16,700 tokens per second, a gain of 23.9 percent, with latency per token falling from 17.7 to 13.9 milliseconds; the gain replicated across a rebuilt environment within 0.05 percent, and a hardware trace confirmed the smaller block kernel executing on the device. A sweep found a flat optimum: blocks of 128 tokens measured 16,805, essentially tied with 256, so the recommendation is robust across that range and the stock 2,048 was the only poor choice for this regime. At 512 concurrent requests the tuned kernel reached 19,511

tokens per second, and the batch scaling step from 256 to 512 improved from 1.13 to 1.17 times, the signature expected when the one batch linear cost becomes cheaper. Two qualifications travel with the result. Profiled runs of the tuned kernel appear slower, because the smaller block multiplies the number of traced kernel invocations and the tracing overhead with it, a measurement artifact that briefly inverted the conclusion and was resolved by the rule of never quoting throughput from a profiled run. And the tuning is regime specific: a long context workload fills the large blocks, would not benefit, and could regress, the same lesson this study met in speculative decoding, now measured from the winning side.

Configuration	Tokens per second	Latency per batch (ms)	Energy bound (mJ per token)
Stock kernel, 256 requests (same session control)	13,479	17.7	15
Tuned kernel, block 256, 256 requests	16,700	13.9	12
Tuned kernel, block 128, 256 requests	16,805	13.9	12
Tuned kernel, block 256, 512 requests	19,511	24.1	10

Table 4. Attention kernel block size tuning through the validated harness, profiling off, treatment verified in the initialization log and by the compiled kernel name in the trace.

6.2 Observations on the compilation model

The TPU serving stack compiles the model ahead of time for a fixed menu of input sizes, called buckets, covering the possible counts of requests and tokens per step. This design delivers the near zero host overhead measured in Section 4, and it worked correctly under scrutiny: a hypothesis that the engine was running steps in an oversized bucket and wasting half its feed forward compute on padding was raised from an early reading of the traces, tested by resolving every traced operation to its true tensor shape, and found to be false; the engine had selected the correct bucket throughout. This dead hypothesis is reported deliberately, because the diagnostic that killed it, attributing every operation to its actual shape rather than trusting names or averages, is reusable and cheap.

The costs of the compilation model are also worth stating. Engine startup, which includes compiling the full bucket menu, ranged from roughly two to five minutes depending on cache state, and each new maximum batch size adds compilation work; irrelevant for long running servers, a real consideration for elastic infrastructure. The compile cache reduces repeat costs within an environment but does not persist across rebuilt environments in the hosted setting used here. The precision experiment in Section 5.3 illustrates the model's rigidity: the specialization that makes steady state execution fast also makes point changes to the compiled pipeline hazardous, so optimization on this stack is best pursued above the compiler, through configuration and workload shape, or below it, through whole kernels, rather than by editing the middle.

Five specific gaps between the stack's advertised and implemented surface were documented, from the quantization findings of Section 5.2 to inefficiencies in the selection and output pipeline, including the observation that requesting the permissive value of a sampling parameter still pays that parameter's full computational cost. All five have been prepared as reports for the maintainers; a new stack improves at the rate its users measure it.

6.3 Compile time: Pallas versus XLA on the same operation

A separate microbenchmark compared the compilation cost of the two routes to a TPU kernel: the same attention computation expressed once as high level operations compiled by XLA, and once as a hand written Pallas kernel lowered through the Mosaic compiler, with equivalent results and equivalent execution times once compiled. Over five trials in fresh processes with the compilation cache disabled, the XLA route compiled in a median of 297.5 milliseconds and the Pallas route in 50.7 milliseconds, a factor of 5.9 and a reduction of 83 percent, with narrow spreads on both sides.

Route	Median compile (ms)	Range over 5 trials (ms)	First execution (ms)
Attention as XLA operations	297.5	296.1 to 308.8	1.2 to 1.8
Attention as Pallas kernel (Mosaic)	50.7	49.0 to 52.0	1.1 to 1.4

Table 5. Compilation cost of the same attention computation by route, single shape, five trials. Execution times after compilation are equivalent; the difference is compile cost only.


```

1 !python3 /content/complete_project/compile_bench.py

... /usr/local/lib/python3.12/dist-packages/jax/_src/cloud_tpu_init.py:88: UserWarning: Transpare
  warnings.warn(
  Shapes: q/k/v = (32,8,128,256) bf16

      pure_XLA | lower:      24.6 ms | compile:   309.8 ms | first run:    1.6 ms
Pallas_Mosaic | lower:      38.1 ms | compile:    51.9 ms | first run:    1.2 ms

Pallas compile / XLA compile ratio: 0.2x
Trace written to /content/profile_bench

```

The plausible mechanism is that XLA must search a large optimization and fusion space over the operation graph, while a Pallas kernel encodes the implementation decisions in advance, leaving only the lowering work. The scope of the claim is narrow, one operation at one shape, and it concerns developer iteration cost, not serving throughput. It matters nonetheless: kernel development traverses the edit, compile, and test loop hundreds of times, so a several fold reduction in the loop's fixed cost is a material productivity difference, and it lowers the price of pursuing the custom kernel opportunity identified in Section 6.1.

7. Energy

The chip's power envelope did not change during the study, so the tenfold growth in output translates directly into a tenfold reduction in energy per token. Because the hosted environment exposes no power telemetry, the study uses the vendor reported chip rating of 197 watts as a deliberate upper bound; a peer reviewed survey both sources this figure and notes that real workload draw typically runs 30 to 60 percent below rating. All figures below are therefore conservative ceilings, and the improvement ratios are exact regardless of true wattage, since the constant cancels.

Stage	Tokens per second	Energy per token, upper bound (mJ)	Tokens per joule, lower bound
Start	1,540	128	7.8
Selection fix	3,577	55	18.2
128 requests	9,790	20	49.7
256 requests	13,470	15	68.4
512 requests	15,338	13	77.9
Kernel tuning, 256 requests	16,700	12	84.8
Kernel tuning, 512 requests	19,511	10	99.0

Table 6. Energy per generated token at the chip's rated power. The tenfold improvement holds for any true wattage because the same chip produced every row.

A cross check against independent literature supports the scale of these figures. A published survey estimates roughly 94 joules per thousand tokens for a 7 billion parameter model on this chip class at a batch of 32; this study measured roughly 20 joules per thousand tokens for a 2.5 billion parameter model at a batch of 128. The model here streams roughly 3.5 times fewer weight bytes per token and runs at four times the batch, and those two factors together predict almost exactly the observed gap. The result also carries a point for sustainability planning: batching is an energy optimization, not only a throughput one, since the same silicon at the same rated power produced 4.3 times the tokens between the first and last rows of the batching campaign.

8. Economic Implications

The commercial meaning of the study is easiest to see in tokens per dollar. The chip class used here is publicly priced at 1.20 dollars per chip hour on demand, so throughput converts directly into unit cost.

Stage	Tokens per second	Tokens per dollar (on demand)	Cost per million tokens
Start	1,540	4.6 million	\$0.217
Selection fix	3,577	10.7 million	\$0.093
256 requests	13,470	40.4 million	\$0.025
512 requests	15,338	46.0 million	\$0.022
Kernel tuning, 256 requests	16,700	50.1 million	\$0.020
Kernel tuning, 512 requests	19,511	58.5 million	\$0.017

Table 7. Generation cost at 1.20 dollars per chip hour, on demand pricing. Committed use and spot pricing would lower every row proportionally.

Read as a business statement: the identical hardware bill buys ten times the product. A service generating one billion tokens per day would need roughly 7.5 chips around the clock at the starting configuration and 0.8 at the final one, before redundancy, a difference of roughly 217 versus 22 dollars per billion tokens at on demand rates. For products with per request economics, such as summarization, extraction, or agent steps, this is the difference between a feature that loses money and one that carries margin.

A throughput level near 13,000 tokens per second is particularly valuable when the consumer of the tokens is another program rather than a person. Agent to agent communication is the clearest case: multi step agent systems exchange plans, tool results, intermediate reasoning, and structured messages among themselves, multiplying token volume by an order of magnitude or more per user task, with no human waiting on any individual token. There, per token latency loses most of its meaning and aggregate throughput becomes the binding constraint, exactly the quantity this study maximized. The same logic covers batch document processing, synthetic data generation, large scale evaluation of other models, and retrieval index enrichment: for these, the frontier in Table 1 should be read from the high batch end, and a single chip at the measured operating points can sustain the internal message traffic of a substantial agent fleet.

Two qualifications keep the claim honest. First, these are offline batch numbers on a decode heavy, short context workload; a production service with irregular arrivals, longer prompts, and latency targets will operate below the maximum, which is why Table 1 reports the full frontier rather than one number, and why 256 requests, not 512, is the planning figure for interactive services. Second, none of the gain required engineering exotica: one configuration correction and one batching campaign, both reachable by any team willing to profile before tuning. The economic finding is less that this chip is cheap and more that unmeasured serving configurations routinely forfeit most of the hardware already being paid for. The latency columns carry their own economic content: batch time to first token grew from 84 to 307 milliseconds across the campaign, and per token latency roughly quadrupled between smallest and largest batch. A chat product sells responsiveness and should buy it with a smaller batch; a nightly pipeline sells volume and should run at the maximum. The contribution here is the measured frontier on which that choice can be made deliberately.

9. Scope and Limitations

The study's claims are bounded, and the bounds are part of the result. The workload was an offline batch of short, repeated prompts with 128 token outputs under deterministic selection, a favorable regime for decode throughput. Services with long documents, streaming arrivals, or sampling requirements will land below these numbers, though the method for finding their own frontier is identical. Prompt caching was enabled and, with repeated prompts, makes the prompt processing phase inexpensive; the figures are decode dominated and affected only marginally, but replications should note the setting. The hardware scope is one chip; production TPU deployments run pods of many, and for a model of this size replication across chips scales near linearly, so the single chip figures are best read as the unit economics of a fleet. The software scope is one version of a fast moving stack, and several findings, in particular every quantization result, are version findings a future release may change. Power figures are rated bounds, not measurements, because the environment exposes no telemetry, and output quality was verified by determinism and inspection rather than formal evaluation, sufficient because no accepted intervention altered the model's computation.

10. Why the Winning Strategies Look Simple

Stated as a headline, the two interventions that produced the tenfold gain sound obvious: use direct token selection and increase the batch. That reading mistakes the fix for the diagnosis. Before the

first profile, the reasonable expectations pointed elsewhere: transformer folklore locates the bottleneck in attention, the optimization literature recommends speculative decoding and quantization as the next steps, and nobody's checklist contains the item that actually held 55 percent of this chip, a selection routine executing 32 search iterations for a parameter value that made the search a mathematical no op. The intervention was one line; finding the line required reading the hardware's own account of where time went, against expectation.

The stronger evidence is the counterfactual, because this study ran the fashionable strategies and measured them. A team following the mainstream sequence without profiling would have deployed speculative decoding, measured here at six times slower in this regime; pursued quantization, measured at zero functioning routes on this stack, one failing silently; or hand optimized a genuine numeric inefficiency, measured at 14 percent slower when removed at the wrong layer. The simple strategies did not win by default. They won an elimination against the sophisticated ones, on the same chip, under the same protocol, and the sophisticated ones lost by measurable margins. Attachment to mainstream techniques is not a neutral preference; on this workload it carried a price this study can state in tokens per second.

Nor is the batching result the folklore version of itself. The folklore says larger batches amortize weight reads. The version practiced here predicted, per operation, which costs would stay flat and which would grow, verified the prediction at three operating points across an eightfold range, located the knee where arithmetic intensity crosses the chip's 240 FLOPs per byte, and closed the decode step's time budget to within a few percent. The knob is common; the accounting is not, and the accounting is what made it safe to stop: the campaign ended at the hardware's measured roofline, not at the end of the author's ideas. The transferable claim is therefore not that simple techniques beat advanced ones in general. It is that in a system whose true costs are unmeasured, technique choice is guesswork with a bias toward fashion, and one profile costs less than one wrong deployment.

The kernel result that followed completes the argument. The same sophisticated instrument, a custom Pallas kernel, that would have been a fashionable first move produced its 24 percent only as a last move, applied to the one operation the profiles had isolated, with the specific mismatch, block sizes tuned for long histories serving short ones, identified from the trace before any code was written. The technique was never the differentiator; the measurement that aimed it was.

11. Practical Implications: Specialized Agents and the Deterministic Path

The configuration that produced these numbers, deterministic selection at high batch, invites a question worth answering carefully: does removing temperature and top_p sacrifice something that fine tuned or specialized models need? The answer runs in the opposite direction, and seeing why connects every part of this study into one architecture. Language models are trained and fine tuned under a cross entropy objective with teacher forcing: the optimization pushes probability mass onto the correct next token. Deterministic selection, taking the highest probability token, is the purest consumption of exactly that objective. Sampling parameters exist to deviate from the trained distribution on purpose, injecting diversity for open ended tasks such as creative writing or brainstorming. They are a feature of generalist use, not a requirement of the model. Fine tuning therefore does not suffer under deterministic decoding; fine tuning is what makes deterministic decoding lossless. A model fine tuned to a narrow task develops a sharply peaked output distribution on that task, with most of the probability mass on the top token, so the behavioral gap between sampled and deterministic decoding shrinks toward zero exactly as specialization increases. A generalist chat model gives up genuine variety at temperature zero; a model fine tuned to emit tool calls, structured JSON, routing decisions, classifications, or extractions gives up almost nothing, and gains determinism, which in production systems is a property with independent value: runs become reproducible, failures become debuggable, outputs become cacheable, and behavior becomes testable.

This reframes the study's first intervention. What was removed was not sampling as a concept but a specific implementation cost: a binary search over a 256,000 entry vocabulary executing 32 iterations per step for a top_p value that made the search a no op, plus penalty transforms. For workloads that genuinely need occasional stochasticity, a temperature only configuration without top_p may retain much of the throughput, since categorical sampling requires no search; this study did not measure that configuration, and it is stated here as an open question rather than a claim. What the study did establish is that for the deterministic path, the entire selection cost disappears, leaving a step that is pure model computation.

The pieces then interlock causally. Specialization through fine tuning makes deterministic decoding lossless for the task. Deterministic decoding removes the selection machinery from the step. The remaining step is dominated by weight streaming, which batching amortizes nearly for free until the arithmetic intensity threshold of Section 4.1. The result, 13.5 to 15.3 thousand tokens per second on one chip, matches the consumption profile of exactly one class of systems: multi agent architectures, where the reader of the tokens is another program, per token latency carries little meaning, and the

volume of tokens per user task is multiplied by an order of magnitude or more of internal traffic in plans, tool results, intermediate reasoning, and structured messages between agents. Even the parts of the study that appeared incidental join this picture. Prefix caching, enabled throughout, is at its most effective in agent fleets, because agents share long system prompts and accumulated context, so the prompt processing cost of inter agent messages collapses. Determinism makes agent pipelines idempotent: the same input state produces the same action, which turns debugging from replaying probabilities into replaying facts. And the model scale studied here, two and a half billion parameters, is the natural size for role specialized agents, where a fleet of small specialists, each fine tuned to one function such as routing, extraction, validation, or summarization, replaces a single large generalist at a fraction of the serving cost.

The economics close the loop. The historical objection to multi agent architectures is that internal chatter multiplies token costs. At the measured operating points, 40 to 46 million tokens per dollar and 13 to 15 millijoules per token as an upper bound, that objection largely dissolves: the internal traffic of a substantial agent fleet fits inside one chip at on demand pricing measured in cents per hour of conversation between agents. A pipeline of specialized two billion parameter agents served this way can be simultaneously cheaper, faster in aggregate, more reproducible, and easier to evaluate than a single frontier model performing the same decomposed task, provided the decomposition itself is sound. The serving side of that trade is what this study measured; it is no longer the uncertain part.

Three boundaries keep this implication honest. First, the training side of the chain, that fine tuned specialists reach adequate task quality at this scale, is architecture this study did not measure; the claim established here is that when they do, the deterministic high batch configuration serves them at these costs with nothing lost to the decoding choice. Second, some agent techniques intrinsically require sampling, such as self consistency voting, best of n generation, and diverse brainstorming; under a purely deterministic configuration a retry returns the identical output, and the available mitigations, prompt perturbation or the unmeasured temperature only path, are workarounds rather than equivalents. Systems built on those techniques should measure the sampling configuration they actually need before assuming these numbers. Third, penalty transforms exist partly to suppress repetition, and deterministic long form generation can loop; the short structured outputs typical of agent messages largely avoid this failure mode, and task fine tuning suppresses it further, but the boundary is real and belongs in any deployment checklist. Within those boundaries, the conclusion stands: the configuration that looked like a benchmark simplification is the correct production configuration for an entire class of systems, and the tenfold gain measured here is the serving cost model for building them.

12. Conclusions

A single mid tier accelerator chip, running an unmodified open model on an open serving engine, moved from 1,540 to 19,511 tokens per second, a factor of 12.7, through measurement driven configuration and one kernel level change delivered through a validated Pallas harness. The recommended operating point delivers roughly 16.7 thousand tokens per second at 13.9 milliseconds per token, at an energy ceiling of 12 millijoules per token and a generation cost near 2 cents per million tokens on demand. The chip finished the campaign more than 99 percent occupied, with its dominant memory operations at 84 to 91 percent of theoretical bandwidth: the study stopped at the hardware, not at a convenient number. The remaining headroom is mapped, not speculative: further attention kernel work beyond the block size retuning that already returned 24 percent; weight quantization, blocked only by stack maturity; and an output pipeline inefficiency worth several percent once fixed at the right layer.

The transferable findings are three. Profile before optimizing, because the largest cost in this system was not where transformer folklore said it would be. Verify that changes took effect, because this stack accepted three configurations it did not apply, and only direct measurement told the difference. And treat every published speedup as a claim about a regime, because the technique measured here as a six times loss is a legitimate win in the regime its advocates measured. The procedures, code, kernel integration techniques, and patch methodology are not disclosed in this document.

Part2: Methods and Implementation: Optimizing LLM Inference on a Single TPU v5e Chip

The Technical Companion to the Measurement Study From 1,540 to 19,511
Tokens per Second

*Strategy by strategy procedures, code paths, patches, and verification for Gemma 2B served
with vLLM on the JAX and Pallas stack*

1. Purpose and Scope

This volume is the study's methods paper. A companion document reported the results: a single TPU v5e chip serving the unmodified Gemma 2B model through vLLM moved from 1,540 to 19,511 tokens per second, with the energy ceiling per token falling from roughly 128 to 10 millijoules and the generation cost from roughly 4.6 to 58.5 million tokens per dollar. This volume discloses how, strategy by strategy: the exact configurations, the code that was written and patched, the profiling procedures that motivated each change, the verification that each change had taken effect, and the outcome of each, including the failures. It is written for machine learning engineers and JAX developers; hardware background helps but is not assumed.

Seven strategies were implemented. Three succeeded: correcting the token selection path (2.3 times), scaling the batch (4.3 times) and tuning the size of the kernel block (a further 1.3 times). Four are reported as instructive failures or validations: speculative decoding, a Pallas kernel transplant, three quantization routes, and a numeric precision patch. Every strategy followed the same loop: profile, predict a specific gain, change one variable, verify the change took effect, and measure against the prediction.

2. Environment, Stack, and Measurement Harness

All work ran in a hosted Colab notebook attached to a single TPU v5e chip (v5e-1, one chip, two cores, 16 GB of high bandwidth memory at 819 GB per second, 197 teraFLOPs per second peak in bfloat16). The software stack was vllm-tpu 0.23.0 with its tpu-inference plugin, JAX 0.10.1, libtpu 0.0.41, and torch 2.10 at the final sessions; earlier sessions resolved older versions, and the engine version line printed at startup was checked whenever numbers from different sessions were compared. One stack detail shapes everything that follows: Gemma 2B is not among the plugin's JAX native model definitions, so the engine loads the vLLM native PyTorch definition and executes it through torchax, the bridge that lowers PyTorch operations to JAX and then XLA. Custom kernels on this stack are written in Pallas and lowered through the Mosaic compiler; the stock attention kernel is a Pallas kernel called ragged paged attention, version 3.

Measurement used one benchmark script for every run, holding the protocol fixed while environment variables selected the treatment: SPEC_DECODE, BATCH_MULT, PROFILE, MODEL, QUANT, and KV_DTYPE. The script builds the engine, times engine initialization separately, runs a small warmup generation to trigger first touch compilation paths, measures batch time to first token by generating exactly one token for the full prompt set, and then runs the main generation of 128 tokens per request under temperature 0.0. The workload is four short prompts repeated BATCH_MULT times. Inter token latency is derived by subtracting the measured time to first token from the main run and dividing by the tokens per request. Two rules were absolute: headline throughput comes only from runs with the profiler off, because tracing adds overhead, and every claimed effect requires a verification independent of the flag that requested it.

Profiling used the engine's start_profile and stop_profile calls wrapped around only the main generation, so traces contain the steady state and not initialization. Traces were read two ways.

TensorBoard's XProf plugin provided the HLO operation statistics table, ranking every compiled operation by device time with its full HLO text, tensor shapes, occurrence counts, and source file attribution; traces were staged into a fresh log directory per analysis to avoid the stale trace trap, in which TensorBoard silently displays a previous session's data, and every table was authenticated by checking that its tensor shapes matched the batch under study before any number was read. When the table's filtering failed, the compressed trace JSON written next to the .xplane file was parsed directly in Python, aggregating operation durations by name and, critically, by the tensor shape extracted from each event's long name. One environment quirk is worth recording for reproducibility: on this stack the profiler ignored the requested output directory and wrote traces under /content/None/plugins/profile, so traces were located by searching for .xplane files rather than trusting the configured path.

Harness variable	Role	Values used
BATCH_MULTI	scales the four prompt set to the target concurrency	8 to 128 (batch 32 to 512)
SPEC_DECODE	enables the ngram speculative configuration	0 or 1
PROFILE	wraps only the main generation in start_profile and stop_profile; runs with 1 are never quoted for throughput	0 or 1
MODEL	checkpoint path, enabling A B runs against local quantized checkpoints	google/gemma-2b or local path
QUANT	the engine's quantization argument, empty means none	empty, fp8
KV_DTYPE	the engine's kv_cache_dtype argument	auto, fp8

Table 1. The benchmark harness switches. One script, one protocol, one variable changed per run. The engine configuration also fixed max_model_len 2048, gpu_memory_utilization 0.75, and tensor_parallel_size 1 throughout.

The engine's own efficiency features were left at their defaults and verified rather than tuned, because the profiles showed they were already doing their job. Asynchronous scheduling overlaps host side work with device execution; the measured gap between decode steps was 85 microseconds against an 11.9 millisecond step at batch 128, under one percent idle, so no host side optimization was warranted. Chunked prefill ran with its default budget of 8,192 tokens per pass, visible in the traces as the 4,096 token shape population. Prefix caching was enabled, and with a repeated prompt set it makes the prompt processing phase inexpensive; this is declared as part of the workload definition since the numbers are decode dominated. Paged attention managed the key value cache in 128 token blocks, 3,249 blocks and 415,872 cacheable tokens under the 0.75 memory fraction, capacity that was never the binding constraint (batch 512 used roughly 72,000 tokens). The optimization surface that remained, and that the six strategies address, was therefore the workload configuration and the compiled operations themselves, not the serving loop.

3. Strategy 1: The Token Selection Path

The first profile of the default configuration contradicted the expected picture. The top two operations by total device time were fusions attributed to a binary search source file inside the sampler, together holding roughly 55 percent of the device, while the transformer's matmuls and the attention custom call divided the remainder. The cause was the request configuration: the baseline used sampling with top_p and repetition penalties, and on this stack a top_p request routes every step through an implementation that performs an iterative binary search over the 256,000 entry vocabulary distribution. The search executes its full fixed iteration count, 32 iterations, even when top_p is 1.0 and the operation is mathematically a no op; the penalty transforms add further per step tensor work over the vocabulary. The model itself was the minority of the step.

The change was one line in the request, not in the engine: SamplingParams(max_tokens=128, temperature=0.0), which selects greedy decoding and bypasses the sampling transforms entirely. Verification was the disappearance of the binary search fusions from the post change HLO table, with the weight streaming matmuls promoted to the top ranks: after the fix, the fused gate and up projection family led the table at roughly 36 percent of device time, the output projection near 17

percent, and attention near 18 percent, the anatomy of a memory bound decode. Throughput rose from roughly 1,540 to 3,577 tokens per second at 32 concurrent requests, a factor of 2.3, and the internal engine counter agreed with the wall clock measurement within a few percent, the paired verification used for every headline number afterward. The observation that a permissive parameter value still pays the full computational cost was documented for the stack's maintainers. The general procedure transfers: profile with the exact SamplingParams of production, because the selection path is part of the workload, and a team whose product genuinely requires sampling should measure that configuration rather than assume the greedy numbers.

4. Strategy 2: Speculative Decoding

Speculative decoding was configured through the engine's native support: method ngram, num_speculative_tokens 5, prompt_lookup_min 2, prompt_lookup_max 4, meaning a lookup mechanism drafts up to five tokens from n gram matches in the existing text and the model verifies them in one pass. At 32 concurrent requests this produced 607 tokens per second against the 3,555 greedy baseline of the same session, a loss of a factor of six.

The mechanism is arithmetic, not implementation failure. Verification of k drafted tokens makes the forward pass process k plus one tokens per request per step; at batch 32 that is up to 192 token positions per step through a memory bound model, and every rejected draft is discarded work. The engine's internal counters corroborated the wall clock number, ruling out a measurement artifact, and the configuration itself was confirmed active in the startup log, so this is a genuine regime result, not an inert flag. Speculation converts spare compute into lower per request latency, which is profitable only when the accelerator has spare capacity, meaning very small batches; the published three to four times claims for the technique come from that regime. It was not pursued further because the study's objective was throughput at meaningful batch sizes. The one run cost a few minutes and prevented a wrong turn, and its per token energy figure, 325 millijoules against the 55 of the greedy baseline it replaced, is the study's clearest illustration that a mismatched optimization is also an energy regression.

5. Strategy 3: Batch Scaling and Compile Buckets

The post fix profile showed the step dominated by operations whose cost is set by streaming fixed bytes: the fused gate and up projection reading a 2048 by 32768 bfloat16 weight (134 MB per layer, measured at about 91 percent of theoretical bandwidth while executing) and the output projection over the 256,000 entry vocabulary reading roughly 1 GB (about 84 percent). Fixed cost per step means near free amortization, so the batch was doubled repeatedly: max_num_seqs in the engine configuration raised the concurrency cap, and BATCH_MULT scaled the prompt set to fill it. Each operating point required both, and each new maximum request count triggered fresh ahead of time compilation.

That compilation model matters operationally. The engine precompiles a fixed menu of input sizes: token paddings of 16 through 8192 and request paddings of 8 through the configured maximum, plus separate sampling, logprob, and placeholder substitution graphs. Cold initialization ranged from 106 to 285 seconds depending on the JAX compile cache state; the cache persists within a virtual machine but not across rebuilt Colab sessions. Runtime steps then execute in the smallest precompiled bucket that fits, with host overhead measured at 85 microseconds per 11.9 millisecond step, under one percent.

Concurrent requests	Tokens per second	Latency per batch (ms)	Batch TTFT (ms)	Energy upper bound (mJ per token)	Tokens per dollar
32	3,577	8.4	84	55	10.7 million
64	6,449	8.9	141	31	19.3 million
128	9,790	11.9	157	20	29.4 million
256	13,470	17.7	187	15	40.4 million
512	15,338	31.2	307	13	46.0 million

Table 2. The batch ladder. Energy uses the 197 watt rated chip power as an upper bound; cost uses 1.20 dollars per chip hour on demand. Scaling factors per doubling: 1.81, 1.50, 1.38, 1.13.

Each rung was verified at the operation level by tracing and reading average times per HLO operation. Three families told the whole story, and Table 3 gives their ladder. The gate and up projection stayed flat while memory bound, then rose as its arithmetic intensity crossed the chip's critical 240 FLOPs per byte between batch 256 and 512, where the 197 teraFLOP roofline predicts 349 microseconds against the 375 measured. The output projection behaved identically at larger scale. The attention custom call scaled linearly, a factor of 7.97 across an 8 times batch increase, because each request reads its own key value cache and the operation's intensity is fixed at about 8 FLOPs per byte, the ratio of query heads to key value heads in this model (Gemma 2B uses multi query attention: eight query heads of dimension 256 share one key value head; the cache is paged in 128 token blocks, 3,249 blocks in this configuration).

Operation (HLO family)	Batch 64 (us)	Batch 128 (us)	Batch 512 (us)	Roofline check at 512
Gate and up projection, 2048 by 32768 bf16 per layer	180.5 to 183	183.5	375	compute bound: predicted 349
Output projection, 2048 by 256000 (embedding tied)	1,526	1,661	2,797	compute bound: predicted 2,725
Ragged paged attention custom call, per layer	89.6	179.1	714.4	memory bound at every batch
Logits fp32 convert plus copy (batch by 256000)	small	700 total	2,799 total	batch linear data movement

Table 3. The operation ladder across the batch campaign, average device time per invocation from HLO statistics, profiled runs only. Occurrence counts matched 18 layers times the decode step count in every trace, the authenticity check applied before reading any number.

At batch 512 these families account for the step almost exactly: attention 18 layers times 714 microseconds is 12.9 milliseconds (41 percent of the 31.2 millisecond step), the gate and up family 6.8 milliseconds (22 percent), the down projection family 4.4 milliseconds (14 percent), and the logits pipeline, matmul plus conversion plus copy plus argmax, near 5.6 milliseconds (18 percent), summing to 30 of 31.2 milliseconds. The batch linear conversion and copy of the logits, absent from folklore but second and third in the ranked table, became Strategy 6.

One trap in this analysis is worth disclosing in detail because it produced a wrong intermediate conclusion. Aggregating trace events by operation name suggested the decode feed forward ops ran with a 1024 token shape, implying half of each step was padding at batch 512. Resolving every event to the tensor shape in its own HLO text disproved it: the population with 258 occurrences (the decode steps) carried shape 512 by 1 by 32768 at 375 microseconds, while the 1024 shaped instances occurred 18 times, once per layer, belonging to a single prefill pass; further shape populations at 16, 256, and 4096 tokens belonged to warmup and chunked prefill buckets. Compiled programs reuse operation names across bucket sizes, so aggregation must key on shape, and occurrence counts must be checked against the expected step count. The corrected analysis confirmed the engine had selected the correct bucket throughout, and the planned `max_num_batched_tokens` intervention was cancelled before consuming any accelerator time.

5.1 What batch scaling did to compilation

Batch scaling is the strategy that interacts most with the ahead of time compilation model, so its costs are itemized here. Raising `max_num_seqs` changes the request padding menu (8, 16, 32, up to the configured maximum) and the attention request padding, so every new operating point paid a fresh precompilation of the backbone across all token buckets, the sampling graphs across all request buckets and flag combinations, the logprob graphs, and the placeholder substitution graphs, roughly 120 compilation tasks in the batch 512 configuration. Within a session the JAX compile cache made repeat initializations several times cheaper (285 seconds cold against 106 warm in the final sessions); across rebuilt virtual machines everything recompiles. None of this touches steady state throughput, which is why initialization was timed separately and excluded from every quoted number, but it defines the operational envelope: on this stack, changing the maximum batch is a deployment event, not a runtime knob. The Pallas attention kernel itself compiles through Mosaic

once per bucket inside this same pass and was never the visible cost; the multi minute totals are dominated by the XLA side of the menu, consistent with the compile time microbenchmark of Section 10.

6. Strategy 4: The Pallas Kernel Transplant

To make the attention kernel modifiable, the stock ragged paged attention v3 source files (kernel.py, kernel_hd64.py, the tuned block size tables, and util.py) were copied out of the installed tpu_inference package into a project directory, my_rpa, made importable with an empty __init__.py and placed on PYTHONPATH. A small module, imported by the benchmark script before the LLM constructor runs, injects the copies back into the live engine: it enumerates the callables defined in the stock kernel module, replaces each with the copy from my_rpa via setattr on the module, and additionally scans sys.modules for any tpu_inference or vllm module holding a direct reference to a replaced function, rebinding those references as well. Import order is the mechanism: the substitution must complete before the engine builds and precompiles, so the injected functions are what the compiler lowers.

Two wrapping rules mattered. Public entry points are wrapped with a functools.wraps probe that prints once on first invocation, giving runtime proof of execution from inside the engine core process. The inner kernel bodies, whose names begin with an underscore, are installed unwrapped, because pallas_call inspects the kernel function it is handed to build the grid and block specification, and a wrapper interferes with that lowering. Verification was double. At runtime, the probes printed during the engine's precompilation pass. In hardware traces, the attention custom call, named for its block configuration (RPA with a one token query block and 2048 token key value blocks in this regime), carried source attribution my_rpa/kernel.py rather than the installed package path, across multiple sessions, batch sizes, and one stack version upgrade. That upgrade is itself a finding about the technique's robustness: the newer tpu-inference resolved the kernel through the module attribute at call time, so the rebinding scan found zero direct references, yet the probes still fired, confirming the module level setattr alone carried the substitution.

The measured result of the unmodified transplant was, by design, statistically identical throughput to stock; equality was the success condition, because a substitution that changed results before any intentional modification would be measuring integration error. Because the vendored kernel compiles through Mosaic inside the engine's own precompilation pass, the substitution added no observable compile time and changed no HLO beyond the source attribution. This validated harness, an editable copy of the production attention kernel proven live, is the platform for the kernel optimization the results volume identifies as the main remaining opportunity: the kernel runs at roughly one quarter of memory bandwidth in this short context regime, dominated by per request overhead rather than data movement, and grows to about 41 percent of the step at batch 512. Strategy 7 reports that harness used for its purpose.

7. Strategy 5: Three Quantization Routes

With the dominant operations at 84 to 91 percent of bandwidth, halving weight bytes was the remaining structural lever. Three routes were attempted, and each terminated at a different layer of the stack. The verification signal for all three was the engine's own initialization log: the model memory line (4.67 GiB for bfloat16 weights) and the key value cache capacity line (415,872 tokens at 16 bit). A working weight quantization must roughly halve the first; a working cache quantization must roughly double the second.

Route	Mechanism attempted	Outcome	Evidence
Integer 8 bit weights	Checkpoint quantized with llmcompressor (scheme W8A16, Linear targets, lm_head excluded), loaded as compressed tensors	Explicit rejection at load	NotImplementedError from the plugin's quantization registry; implemented methods were None and fp8 only
Float 8 bit weights	quantization set to fp8 on the standard checkpoint	Explicit failure, unhelpful message	KeyError on quant_method: the path requires a checkpoint quantized in advance and has no on the fly mode

Route	Mechanism attempted	Outcome	Evidence
Float 8 bit KV cache	kv_cache_dtype set to fp8	Silent no op	Engine ran normally; cache capacity line unchanged at 415,872 tokens, allocation still bfloat16

Table 4. Quantization attempts on tpu-inference 0.23 for a model on the PyTorch fallback path. Zero routes functioned; each failed at a different layer with a different loudness.

Source inspection explained the pattern. The platform's supported quantization list advertises several methods (compressed tensors, awq, fp8, and others), but the JAX quantization registry that the loader actually consults implements two, none and fp8, and the PyTorch fallback loader carries scaffolding for quantized layers with a comment stating that only unquantized linear layers are handled for now. The integer route's checkpoint was produced with llmcompressor's one shot flow, scheme W8A16 over Linear targets with lm_head excluded as the conservative first configuration (in this model the output projection shares the embedding weights, the riskiest layer to quantize); the checkpoint remains valid for GPU serving even though this stack rejected it. One procedural trap deserves a warning: a published quantized checkpoint with a nearly identical name targets Gemma 2, a different architecture from Gemma 2B, and benchmarking it would have silently invalidated every baseline; quantized checkpoints must match the exact base model of the campaign. The silent cache case is the instructive failure: the flag traveled through the whole configuration surface, appeared in the engine's printed configuration, and changed nothing, which is why the study's rule is to verify allocations and memory lines rather than flags. One run in the sequence also demonstrated the rule in reverse: a treatment run whose throughput matched baseline was traced to the environment variable never reaching the engine (the script predated the flag), so its number was reclassified as a third baseline replication rather than evidence about quantization. All findings were prepared as reports for the maintainers; they are version findings that a future release may change.

8. Strategy 6: The Output Precision Patch

The batch 512 trace ranked a 32 bit conversion of the logits tensor (f32, batch by 256000) and a subsequent copy of the same tensor at positions two and three of the whole table, 2.8 milliseconds per step combined, batch linear, sourced to one line in the runner. Reading the code found the conversion in two places: the runner converts the logits to float32 before calling the sampler, and the sampler converts again internally. The decisive observation came from the sampler's own ordering: the greedy token is computed by argmax on the original bfloat16 logits before either conversion executes, so under deterministic selection the 32 bit precision feeds only a returned tensor the benchmark never consumes, and since a widening conversion cannot change which element is largest, removing it cannot alter any output.

The patch made both conversion sites conditional on the sampler metadata's do_sampling flag, a Python level condition that is static at trace time, so the compiler builds separate graphs per case with no runtime branch: the sampling path stays byte identical, and only greedy decoding skips the conversions. Application used anchored text replacement into the installed package with a uniqueness assertion on the anchor, the same mechanism as the kernel transplant; both anchors matched exactly once, confirming the code was as read. The prediction was a gain of three to seven percent.

The measurement was the opposite: 11,641 tokens per second on the first patched run and 8,839 on the second, against the 13,470 baseline, with batch time to first token multiplied by five (187 to roughly 995 milliseconds), reproducible on a warm compile cache and unstable between runs, the worst combination. The patch was reverted by the inverse replacement and the baseline reconfirmed. The probable mechanism is that the engine's precompiled pipeline, including the graphs built during its warmup pass, was specialized around 32 bit logits; changing the returned dtype at one point pushed downstream consumers (output copies, logprob machinery, the runner's own precompiled sampling graphs) off their precompiled paths on every step. The lesson is stated as a rule for this class of stack: operation level arithmetic does not compose across just in time compilation boundaries, so a cost measured inside one compiled program cannot simply be subtracted by editing that program's boundary. The inefficiency itself remains real and was documented for the maintainers, where the fix can be made at the pipeline level with all consumers rekeyed together. On a hosted virtual machine, edits to installed packages do not survive an environment rebuild, which makes both the experiment and its revert cheap and fully reversible.

9. Strategy 7: Attention Kernel Block Size Tuning

The final strategy used the transplant harness of Strategy 4 for its intended purpose. The target came from the traces: the attention kernel ran at roughly one quarter of memory bandwidth and had grown to the largest cost at scale. The kernel's compiled name encodes its block configuration, and every trace showed key value blocks of 2,048 tokens. Reading the vendored source explained why: the default heuristic sizes blocks to carry about 16 megabytes so that each block on its own saturates memory bandwidth, and under this model's configuration that resolves to 2,048 tokens, the full maximum context. The workload's sequences average about 140 tokens, so roughly 93 percent of every block was masked padding, loaded and processed for nothing.

The patch added an environment variable override inside the vendored copy's block size function for the decode case, leaving stock behavior untouched when the variable is unset. Block sizes are static compile time arguments, so each override value produces its own compiled kernel, and the change carries two built in verifications: a print line at every compilation that uses the override, and the compiled kernel's name itself, which must show the new block size in any trace. Outputs cannot change, because block size affects scheduling, not arithmetic; the generations were confirmed identical to the stock kernel's.

Measured with the full protocol: in a same session comparison at 256 concurrent requests, 13,479 tokens per second stock against 16,700 tuned, a gain of 23.9 percent, with latency per token falling from 17.7 to 13.9 milliseconds, replicated across a rebuilt environment within 0.05 percent. A block of 128 tokens measured 16,805, statistically tied with 256, so the optimum is a plateau and only the stock 2,048 was a poor choice for this regime. At 512 concurrent requests the tuned kernel reached 19,511 tokens per second, and the batch scaling step from 256 to 512 improved from 1.13 to 1.17 times, the expected signature of a cheaper batch linear term. The campaign total became 12.7 times over the starting configuration, with the energy bound at 10 millijoules per token and 58.5 million tokens per dollar at the throughput maximum.

Two findings about method came with the result. First, profiled runs of the tuned kernel measured near baseline speed and briefly inverted the conclusion: the smaller block multiplies the number of traced kernel invocations roughly eightfold, and per event tracing overhead becomes visible at that count. The rule of never quoting throughput from a profiled run is what resolved it; the unprofiled comparison and the trace's kernel names together carry the claim. Second, the tuning is regime specific. The 16 megabyte heuristic is correct for long contexts that fill the blocks; a long context deployment should expect no gain and possibly a regression from this change. It is the speculative decoding lesson measured from the winning side: kernels are tuned for a regime, and the regime must be checked against the workload.

Configuration	Tokens per second	Latency per token (ms)	Verification
Stock kernel (same session control)	13,479	17.7	matches the baseline replication family
Block 256, 256 requests	16,700	13.9	override line; kernel name RPA-d-p_128-bq_1_1-bk v_256_256 in trace; cold replication 16,692
Block 128, 256 requests	16,805	13.9	override line in initialization log
Block 256, 512 requests	19,511	24.1	override line in initialization log

Table 5. Strategy 7 results, profiling off. Every row carries its own treatment verification, and the compiled kernel name is the self labeling proof.

10. Measuring Compile Time: Pallas Versus XLA

The compile cost comparison in the results volume was produced by a standalone script that times the three phases of `jax.jit` separately: lower (tracing the Python function to an intermediate representation), compile (the backend building the executable), and first execution. The JAX compilation cache was disabled for the measurement, and each trial ran in a fresh process so no cached tracing survived between measurements. The same attention computation was expressed twice: as high level operations, two `einsums` with a `float32` softmax between them, compiled by XLA; and as the flash attention Pallas kernel from `jax.experimental`, lowered through Mosaic, on identical

bfloat16 inputs of shape 32 by 8 by 128 by 256. Over five trials the XLA route compiled in a median of 297.5 milliseconds (range 296.1 to 308.8) and the Pallas route in 50.7 (49.0 to 52.0), a reduction of 83 percent, with lowering costs of 24 and 39 milliseconds respectively and first executions equivalent at 1.1 to 1.8 milliseconds. The interpretation is bounded: one operation at one shape, measuring developer iteration cost rather than runtime, and consistent with the mechanism that XLA searches a fusion and optimization space over the graph while a Pallas kernel arrives with its implementation decided, leaving only lowering. For kernel development, where the edit, compile, and test loop is traversed hundreds of times, the several fold difference is the quantity that matters.

11. Per Strategy Economics

Strategy	Throughput (tok/s)	Energy bound (mJ/token)	Tokens per dollar	Verdict
Starting configuration (batch 32, top_p path)	1,540	128	4.6 million	baseline
1. Greedy selection path	3,577	55	10.7 million	kept, 2.3x
2. Speculative decoding (ngram, batch 32)	607	325	1.8 million	rejected, 0.17x
3. Batch scaling to 256 (recommended)	13,470	15	40.4 million	kept, 8.7x total
3. Batch scaling to 512 (maximum)	15,338	13	46.0 million	kept, 10.0x total
4. Pallas kernel transplant	unchanged	unchanged	unchanged	validated harness
5. Quantization (three routes)	unchanged	unchanged	unchanged	blocked by stack
6. Output precision patch	11,641 then 8,839	worse	worse	reverted
7. Attention kernel block size tuning (block 256)	16,700 / 19,511	12 / 10	50.1 / 58.5 million	kept, 12.7x total

Table 6. Every strategy with its measured economic consequence. Energy uses the 197 watt rating as an upper bound; cost uses 1.20 dollars per chip hour. The three kept strategies account for the 12.7 times gain.

The business reading is direct. The identical chip hour, the identical watt budget, and the identical model produce ten times the tokens after two configuration changes, which means fleets, energy budgets, and per token unit costs planned on default configurations are overstated by up to that factor. A service generating one billion tokens per day needs roughly 7.5 chips at the starting configuration and 0.8 at the final one, before redundancy.

Time to first token deserves its own reading, because the strategies moved it in different ways. The selection fix improved it (the prefill's own logits pass no longer pays the sampler search). Batch scaling raised it steadily, from 84 milliseconds at batch 32 to 307 at 512, because the number is defined here as prefill plus one decode step for the entire batch: more requests mean more prompt tokens through chunked prefill and a heavier first step. The failed precision patch multiplied it by five while claiming to optimize a decode cost, the clearest demonstration in the study that time to first token is a sensitive detector of compilation path regressions and belongs in every benchmark printout, not only throughput. Latency prices the choice of operating point: batch 256 holds 17.7 milliseconds per token, suitable for interactive services, while batch 512 trades 79 percent worse per token latency for 13 percent more throughput and suits machine consumers, agent to agent traffic, and offline pipelines. The failed strategies also carry economic content: each was bounded by a single cheap run or a source inspection before it could consume engineering time at scale, and the reverted patch cost one session while producing the study's most transferable engineering rule.

12. Reproduction Notes

Reproducing the study requires four artifacts: the notebook's installation cell (versions resolve at run time and did drift once during the study, from JAX 0.7 to 0.10 with a matching libtpu jump, which the engine's startup version line detects); the benchmark script with its environment variable switches from Table 1; the injection module; and the vendored kernel directory created by copying the installed package's ragged paged attention v3 files. Cell order matters: install, write the project

files, copy the kernel sources, then run, with the injection imported before the engine constructor. The 256 request operating point was replicated in three independent sessions on rebuilt virtual machines at 13,517, 13,470, and 13,484 tokens per second, a spread of one third of one percent; one further session returned 12,991, a 3.6 percent environment dip reported rather than discarded, so replications should expect the headline near 13.5 thousand with occasional session level variance of a few percent.

Three habits carried the study and are the recommended protocol for any similar effort. First, never quote throughput from a profiled run, and never trust an unprofiled claim about where time goes. Second, pair every treatment with an authenticity check that does not depend on the treatment's own flag: a memory line, an allocation size, a tensor shape in a trace, a probe print from inside the engine process; this study caught three silently inert configurations and one mislabeled trace by that habit alone. Third, when aggregating traces, key on tensor shape and check occurrence counts against the expected step count, because operation names are reused across compiled buckets and name level averages mix programs; that discipline both raised and then correctly killed the padding hypothesis of Section 5. The prompt set, the deterministic selection, the offline batch execution, and the enabled prefix cache define the workload regime; all numbers in this volume and the results volume are claims about that regime, measured to the hardware's roofline within it.